

Chapter 3

Unsupervised Learning & Preprocessing

Scaling, dimensionality reduction,
and clustering

Learning Objectives

- Understand what unsupervised learning is and why it's **hard to evaluate**
- Scale features correctly — and avoid the classic **data-leakage** bug
- Reduce dimensions & extract features with **PCA, NMF, t-SNE**
- Group data with **k-Means, agglomerative clustering, DBSCAN**
- Evaluate clusters **with and without** ground truth

Types of Unsupervised Learning

No known output, no teacher — just find structure in the input data.

Transformations

- Create a **new representation** of the data
- Dimensionality reduction (e.g. $\rightarrow$ 2-D for visualization)
- Feature extraction, topic modeling

Clustering

- **Partition** data into groups of similar items
- e.g. group photos by who appears in them

The Central Challenge: Evaluation

- Usually **no labels** → no way to know the "right" answer
- Hard to tell whether a model **learned something useful**
- The photo grouping might split by *pose* instead of *person* — and we can't tell it otherwise

Consequences:

- Often used for **exploration** — to understand data
- Or as a **preprocessing** step that helps a supervised model

Scaling methods don't use labels either → they count as unsupervised.

Part 1

Preprocessing & Scaling

Why Scale?

Some models — **SVMs, neural networks, k-NN** — are very sensitive to feature scales.

- A feature ranging 0–1000 shouldn't dominate one ranging 0–1
- The fix: a per-feature **shift and rescale**
- These are simple **transformers**: `fit` then `transform`

Four Common Scalers

Scaler	What it does
StandardScaler	mean 0, variance 1 (per feature)
RobustScaler	uses median & quartiles → ignores outliers
MinMaxScaler	shifts data into exactly [0, 1]
Normalizer	scales each <i>sample</i> to unit length (direction only)

Same interface — swap one for another by changing the class name.

Applying a Scaler

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(X_train)                # learns per-feature min & range
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

- Unlike classifiers, `fit` uses **only** `x_train` — no `y`
- `transform` returns the new representation (same shape)

Scale Train & Test the *Same* Way

⚠ Fit the scaler on **training data only**, then apply that *same* transform to test.

```
# WRONG — refitting on the test set moves the points incongruously  
test_scaler = MinMaxScaler().fit(X_test)    # DON'T DO THIS
```

- Test values may land slightly outside $[0, 1]$ — that's **correct**
- Think of a single test point: you can't rescale it to $[0,1]$ alone
- Refitting on test = **data leakage** → misleading results

Shortcut: fit_transform

For fitting and transforming the **same** data, use one efficient call:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)    # = fit(X).transform(X), faster
```

Use it on the **training set only** — never on the test set.

Effect on Supervised Learning

Scaling can transform results — same SVM, same data:

```
svm = SVC(C=100)
svm.fit(X_train, y_train)           # unscaled cancer
svm.score(X_test, y_test)          # 0.63

svm.fit(X_train_scaled, y_train)    # MinMax-scaled
svm.score(X_test_scaled, y_test)    # 0.97
```

Use scikit-learn's scalers rather than reimplementing — easy to slip up.

Part 2

Dimensionality Reduction & Feature Extraction

Look at Your Data First

Before any model, inspect the raw features. For cancer, plot **per-feature histograms** split by class (benign vs malignant).

- *"worst concave points"* → histograms **barely overlap** → very informative
- *"smoothness error"* → histograms **overlap** → uninformative
- But a histogram shows **one feature at a time** — it misses **interactions**

PCA captures feature interactions, giving a fuller picture in a single scatter plot.

Principal Component Analysis (PCA)

Rotates the data so the new axes (principal components) are **uncorrelated**, ordered by the variance they capture.

- Component 1 = direction of **maximum variance**; each next is orthogonal
- Keep the top k components → **compress** while preserving structure

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2).fit(X_scaled)
X_pca = pca.transform(X_scaled)
```

⚠ Scale first (StandardScaler) — PCA follows variance.

PCA for Visualization

Plot high-dimensional data in 2-D. Cancer: **30 features** → **2 components**.

```
X_pca = PCA(n_components=2).fit_transform(X_scaled)    # (569, 30) → (569, 2)
```

- The two classes separate well in the 2-D plot → even a linear model could do well
- PCA is **unsupervised** — it never sees the labels; we only *color* by class after

Interpreting the Components

```
pca.components_          # shape (2, 30): each row is one component
```

- Each component is a **combination of all original features**
- On cancer: the first component has **all-positive** signs → features are correlated
- ⚠ Components are usually **hard to interpret** — the axes mix many features

Feature Extraction: Eigenfaces

Apply PCA to the **Labeled Faces in the Wild** dataset (3023 images, 87×65 px, 62 people).

- Raw pixels are a **poor** similarity measure (shift by one pixel → looks totally different)
- 1-NN on raw pixels: only **~27%** accuracy
- PCA with **whitening**, 100 components → 1-NN improves to **~36%**

```
pca = PCA(n_components=100, whiten=True).fit(X_train)
X_train_pca = pca.transform(X_train)
```

Components (eigenfaces) capture contrast, lighting, face alignment.

PCA as Reconstruction

PCA can rebuild data from a few components via `inverse_transform`.

```
X_reconstructed = pca.inverse_transform(pca.transform(X_test))
```

- **10 components** → only the essence (orientation, lighting)
- **More components** → progressively more detail returns
- With as many components as pixels → perfect reconstruction

Useful for **noise reduction** and compression.

Non-Negative Matrix Factorization (NMF)

Like PCA, expresses each point as a **weighted sum of components** — but components **and** coefficients must be **non-negative**.

- Yields **additive, parts-based**, more interpretable components
- Only works on **non-negative data**
- Components have **no natural ordering; random init** → results vary by seed

```
from sklearn.decomposition import NMF
nmf = NMF(n_components=15, random_state=0).fit(X)
```

NMF: Finding Additive Sources

NMF shines on data that is a **sum of sources** — audio, gene expression, text.

Signal separation example: 3 original signals mixed into 100 measurements.

```
S_recovered = NMF(n_components=3).fit_transform(X)    # recovers the 3 sources
```

- NMF recovers the original sources well; **PCA fails** (it wants orthogonal, high-variance directions)
- On faces, NMF components look like **face prototypes** (e.g. faces turned left/right)

Other Decomposition Methods

PCA and NMF both write each point as a **weighted sum of components**. scikit-learn offers more, each with different constraints:

- **ICA** (Independent Component Analysis) — statistically **independent** sources
- **Factor Analysis (FA)** — latent factors plus per-feature noise
- **Sparse coding** (dictionary learning) — few **active** components per point

See the user guide's *decomposition methods* page. The choice depends on the structure you expect in the data.

Manifold Learning with t-SNE

A **nonlinear** method built **for visualization** (usually to 2-D).

- Keeps **nearby points close**, emphasizing neighbor relationships
- Finds structure PCA's rotation-and-drop approach misses

⚠ **No** `transform` — can't be applied to new data. Use `fit_transform` only.

```
from sklearn.manifold import TSNE
digits_tsne = TSNE(random_state=42).fit_transform(digits.data)
```

t-SNE on the Digits Dataset

Handwritten digits (8×8 grayscale):

- **PCA** to 2-D → classes overlap heavily (only 0, 4, 6 separate)
- **t-SNE** → nearly all ten digit classes form **clear, separate groups**
- Remarkable given it uses **no label information** at all

Tuning knobs `perplexity` and `early_exaggeration` usually have minor effects.

Part 3

Clustering

k-Means Clustering

Find k **cluster centers**, alternating two steps until assignments stop changing:

1. **Assign** each point to its nearest center
2. **Recompute** each center as the mean of its points

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3).fit(X)
km.labels_          # cluster index per point (arbitrary numbers)
km.cluster_centers_ # the centroids
km.predict(X_new)   # assign new points to nearest center
```

Must choose `n_clusters` (default 8, no good reason).

Failure Cases of k-Means

k-Means assumes clusters are **convex, equal-sized blobs** — it always splits halfway between centers.

It struggles with:

- Clusters of **different densities**
- **Non-spherical** / stretched clusters
- Complex shapes like **two_moons**

Knowing the right number of clusters doesn't guarantee k-Means will find them.

k-Means as Vector Quantization

Each point is represented by its **single** cluster center — a decomposition view called *vector quantization*.

- Can use **more clusters than features** → a richer representation
- On two_moons, 10 centers give features that make the halves **linearly separable**
- `kmeans.transform(X)` → distances to every center as new features

Scales well; `MiniBatchKMeans` handles very large datasets.

Three Views: PCA, NMF, k-Means

All three express points via components — with different rules:

Method	Each point =	Components point toward
PCA	sum of orthogonal components	directions of max variance
NMF	non-negative sum	additive "parts" of the data
k-Means	one cluster center	representative regions

k-Means is the extreme case: **a single component** per point (vector quantization).

k-Means: Strengths & Weaknesses

- ✓ Simple, fast, **scales** to large datasets; easy to understand
- ✓ Cluster centers **summarize** each group
- ✗ Relies on **random initialization** (scikit-learn runs it 10× and keeps the best)
- ✗ Restrictive assumptions on cluster **shape**
- ✗ Must **specify the number of clusters**

Agglomerative Clustering

Bottom-up: start with each point as its own cluster, repeatedly **merge the two closest** until k remain.

Linkage = how "closest" is measured:

Linkage	Merges clusters with...
ward (default)	least increase in variance
average	smallest average distance
complete	smallest maximum distance

```
from sklearn.cluster import AgglomerativeClustering
AgglomerativeClustering(n_clusters=3).fit_predict(X)    # no predict()
```

Dendrograms

Agglomerative clustering builds a **hierarchy** — visualize it as a dendrogram.

- Each **merge** = a horizontal join; branch **length** = distance bridged
- **Cut** the tree at any height to choose the number of clusters
- scikit-learn can't draw them — use **SciPy** (`ward`, `dendrogram`)

```
from scipy.cluster.hierarchy import dendrogram, ward
dendrogram(ward(X))
```

Still can't separate complex shapes like `two_moons`.

DBSCAN

Density-based clustering: clusters are **dense regions** separated by sparse ones.

Three kinds of points (parameters `eps`, `min_samples`):

- **Core** — $\geq$ `min_samples` neighbors within `eps`
- **Boundary** — near a core point but not dense itself
- **Noise** — everything else, labeled `-1`

```
from sklearn.cluster import DBSCAN
DBSCAN(eps=0.5, min_samples=5).fit_predict(X)
```

Tuning & Using DBSCAN

- `eps` — how close is "close": larger → bigger, fewer clusters
 - `min_samples` — minimum cluster size; larger → more points become noise
 - **Does not need the number of clusters** set in advance
 - Scaling (StandardScaler/MinMaxScaler) makes `eps` easier to choose
- ✓ Finds **arbitrary shapes** (splits two_moons with defaults) and flags **outliers**
- ✗ Slower than k-Means; boundary points can depend on point order

Part 4

Evaluating Clustering

Evaluating with Ground Truth

If you *do* have true labels, compare partitions with:

- **Adjusted Rand Index (ARI)** and **Normalized Mutual Information (NMI)** — both in $[0, 1]$
- Random assignment $\rightarrow 0$; perfect match $\rightarrow 1$

```
from sklearn.metrics.cluster import adjusted_rand_score
adjusted_rand_score(y_true, clusters)
```

On two_moons: random 0.0, k-Means 0.50, agglomerative 0.61, **DBSCAN 1.00**.

Don't Use `accuracy_score`!

Cluster label **numbers are arbitrary** — the same grouping can use different integers.

```
clusters1 = [0, 0, 1, 1, 0]
clusters2 = [1, 1, 0, 0, 1]      # identical grouping, swapped names
accuracy_score(clusters1, clusters2)      # 0.00 ✗ misleading
adjusted_rand_score(clusters1, clusters2) # 1.00 ✓ correct
```

Always use **ARI / NMI**, which only care about *which points share a cluster*.

Evaluating without Ground Truth

Usually there are **no labels** — that's the whole point.

- **Silhouette score** — measures cluster **compactness** (higher = tighter)
- But compact $\neq$ meaningful: on two_moons, k-Means **scores higher** than the "better" DBSCAN result
- **Robustness-based** checks (perturb data, compare results) are more reliable

*In the end, the only sure test is **manual inspection** of the clusters.*

Case Study: Clustering Faces

Applying all three to the eigenface representation of LFW is a **qualitative** process:

- **DBSCAN** (defaults) → labels *everything* noise; raising `eps` yields one big cluster plus a few small ones — the "noise" images are **odd crops / angles** → **outlier detection**
- **k-Means** & **agglomerative** → many **similarly-sized** clusters (they must assign every point)
- ARI between k-Means and agglomerative ≈ 0.13 → they partition quite differently

*There's no accuracy score here — you judge clusters by **looking** at their members.*

Summary of Clustering Methods

Method	Set clusters?	Cluster shapes	Extras
k-Means	yes (k)	convex blobs	cluster centers; vector quantization
Agglomerative	yes (k)	varied	full hierarchy → dendrograms
DBSCAN	no (ϵ)	arbitrary	detects noise / outliers

All three scale to real data and are easy to understand.

Chapter Summary

- Unsupervised learning finds structure with **no labels** — and is **hard to evaluate**
- **Always scale** — and fit the scaler on **training data only**
- **PCA / NMF / t-SNE** compress, extract features, or visualize high-dim data
- **k-Means / agglomerative / DBSCAN** suit different cluster shapes & needs
- Evaluate with **ARI/NMI** (never accuracy); otherwise rely on **inspection**

Next: Chapter 4 — Representing Data & Engineering Features